

727723EUA1109 : Sanjay M

```
In [3]: import numpy as np

def sigmoid(z): return 1 / (1 + np.exp(-z))
def relu(z):    return np.maximum(0, z)
def tanh(z):    return np.tanh(z)

ACTIVATIONS = {"sigmoid": sigmoid, "relu": relu, "tanh": tanh}

def forward_pass(X, layers):
    cache = []
    A = np.array(X, dtype=float)
    for layer in layers:
        Z = np.dot(layer["W"], A) + layer["b"]
        A = ACTIVATIONS[layer["activation"]](Z)
        cache.append({"Z": Z, "A": A, "activation": layer["activation"]})
    return cache

# Architecture: 3 -> 2 -> 2 -> 1 neurons
layers_relu = [
    {"W": np.array([[0.5, -0.2, 0.4], [0.1, 0.3, -0.3]]), "b": np.array([0.1, 0.2]), "activation": "relu"},
    {"W": np.array([[0.3, 0.7], [0.5, -0.1]]), "b": np.array([-0.05, 0.1]), "activation": "relu"},
    {"W": np.array([[0.6, 0.4]]), "b": np.array([0.0]), "activation": "sigmoid"}
]

X = [1.0, -0.5, 2.0]
cache_relu = forward_pass(X, layers_relu)
```

```
In [4]: def print_summary(cache, title="Forward Pass Summary"):
    print(f"\n{'='*50}")
    print(f"  {title}")
    print(f"{'='*50}")
    for i, entry in enumerate(cache, 1):
        Z = np.round(entry["Z"], 4)
        A = np.round(entry["A"], 4)
        act = entry["activation"].capitalize()
        print(f"  Layer {i} | Z = {Z} | A ({act}) = {A}")
    print(f"  Final Output: {np.round(cache[-1]['A'], 4)}")
    print(f"{'='*50}\n")

print_summary(cache_relu, "Layer 2 Activation = ReLU")
```

```
=====
  Layer 2 Activation = ReLU
=====
  Layer 1 | Z = [ 1.5 -0.45] | A (Relu) = [1.5 0. ]
  Layer 2 | Z = [0.4 0.85] | A (Relu) = [0.4 0.85]
  Layer 3 | Z = [0.58] | A (Sigmoid) = [0.6411]
  Final Output: [0.6411]
=====
```

```
In [5]: import copy

layers_tanh = copy.deepcopy(layers_relu)
layers_tanh[1]["activation"] = "tanh" # Only Layer 2 changes
```

```

cache_tanh = forward_pass(X, layers_tanh)
print_summary(cache_tanh, "Layer 2 Activation = Tanh")

# Comparison
print("Comparison of Final Outputs:")
print(f"  ReLU  (Layer 2): {np.round(cache_relu[-1]['A'], 4)}")
print(f"  Tanh  (Layer 2): {np.round(cache_tanh[-1]['A'], 4)}")

```

```

=====
Layer 2 Activation = Tanh
=====
Layer 1 | Z = [ 1.5 -0.45] | A (Relu) = [1.5 0. ]
Layer 2 | Z = [0.4  0.85] | A (Tanh) = [0.3799 0.6911]
Layer 3 | Z = [0.5044] | A (Sigmoid) = [0.6235]
Final Output: [0.6235]
=====

```

```

Comparison of Final Outputs:
ReLU  (Layer 2): [0.6411]
Tanh  (Layer 2): [0.6235]

```